UNIVERSIDAD DEL VALLE DE GUATEMALA

CC3089 - Base de Datos 2

Sección 20

Daniela Mesalles

Proyecto 1

MongoDB

Roberto Barreda, 23354

Javier España, 23

Diego López, 23747

GUATEMALA, 26 de febrero de 2026

Índice

1. **Descripción del Problema**
    a. **Contexto**
    Desde que surgieron las plataformas de pedidos en línea y delivery, el modelo de negocio de muchos restaurantes y la industria gastronómica en general fueron transformados. Estas plataformas no solo se presentaron y se han mantenido, sino que han tenido un crecimiento acelerado que genera una necesidad de gestionar información en tiempo real con sistemas escalables. Se da una interacción constante de usuarios, repartidores y restaurantes con pedidos, consultas y reseñas que se dan geográficamente. Debido a todas estas características, la base de datos de dichas plataformas tiene que tener una arquitectura eficiente, orientada al rendimiento y flexible.
    b. **Problema a Resolver**
    La necesidad de diseñar un sistema de gestión de pedidos y reseñas de restaurantes debe de permitir los siguientes puntos clave:
        - Registrar restaurantes con ubicación geográfica
        - Tener menús
        - Permitir hacer pedidos en línea
        - Manejar pedidos en tiempo real
        - Dejar la redacción de reseñas con calificación y contenido multimedia
        - Consultas:
            - Platos más vendidos
            - Restaurantes mejor calificados
            - Ventas por periodo
            - Restaurantes cercanos a una ubicación en específico

    Todo esto se debe de dar con un buen rendimiento, índices adecuados, escalabilidad horizontal, soporte para archivos grandes (tipo, multimedia) y manejo eficiente de datos y referencias
    c. **Actores y sus acciones**
        i. Usuario
            1. Crear una cuenta
            2. Realizar pedidos
            3. Consultar por ubicación algún restaurante
            4. Dejar reseñas: calificación, comentario y imágenes
        ii. Restaurante
            1. Administrar su menú
            2. Recibir pedidos
            3. Registrar su información
        iii. Sistema
            1. Persistencia de datos
            2. Almacenamiento de archivos usando GridFS
            3. Optimizar consultas con buenos índices
            4. Reportes con aggregation pipelines
            5. Transacciones multi-documento
    d. **Operaciones del Sistema (principales)**
        i. CRUD: Creación, lectura, eliminación y updates de restaurantes, usuarios, menús, órdenes y reseñas.

ii. Analíticas: Calificación promedio por restaurante, platillos más vendidos, análisis de veneta por fecha y consultas para restaurantes cercanos
iii. Transaccionales: Pedidos con varios artículos, updates de estado de pedidos y el registro de órdenes con reseñas vinculadas luego de que la órden haya sido entregada.

e. Aspectos técnico

i. índices multikeys, simples, compuestos, de texto y geoespaciales
ii. Aggregation Framework para análisis de data
iii. Transacciones multi-documento por consistencia
iv. Shard key para escalabilidad horizontal
v. Referencias para relaciones de crecimiento que llega alto
vi. Documentos embebidos para datos dependientes

2. Alcance y Supuestos

Usando MongoDB como motor de base de datos, se mapea un diseño y implementación orientada a pedidos y su gestión y reseñas de restaurantes.

Tabla 1. Alcances del Sistema vs. Fuera del Alcance

In-Scope	Out-of-Scope
Gestión de restaurantes y menús: Consultas por proximidad geográfica, actualización de información y estado del restaurante y menú (precios y disponibilidad), almacenamiento geográfica con GeoJSON, registro de restaurantes y menús con información descriptiva	Proceso de pagos reales
Gestión de usuarios: Manejo de distintos roles (admin, cliente), registro y autenticación de usuarios y asociación de usuarios con pedidos y reseñas	Integración con banca
Gestión de órdenes: creación de pedidos (con varios artículos), cálculo de totales, manejo de estado del pedido y consulta histórica por usuario o restaurante	Sistema de repartidores en tiempo real
Gestión de reseñas: Asociación de reseñas a restaurantes o órdenes, calificaciones numéricas, comentarios textuales, soporte de imágenes con GridFS y rating promedio por cálculo dinámico	Autenticación con proveedores externos (ej. Google)
Análisis y reportes: restaurantes mejor calificados, platillos maps vendidos, ventas por tiempo y busquedas	Infraestructura física de despliegue en producción

a. Supuestos

i. MongoDB Atlas como entorno de base de datos
ii. Sistema como backend API

iii. Se usarán transacciones multi-documento
iv. Data base configurada para requerir índices en sus consultas
v. Escalabilidad horizontal con estrategia de shard key
vi. Imagenes de reseñas almacenadas con gridFS
vii. Se manejan grandes volúmenes de datos

3. Modelo de Datos

El sistema combina referencias y documentos embebidos, optimiza consultas frecuentes y da escalabilidad. Para esto, se usará un modelo orientado a documentos.

Existen 5 colecciones principales:

- Restaurantes
- Usuarios
- Artículos del Menú
- Órdenes
- Reseñas

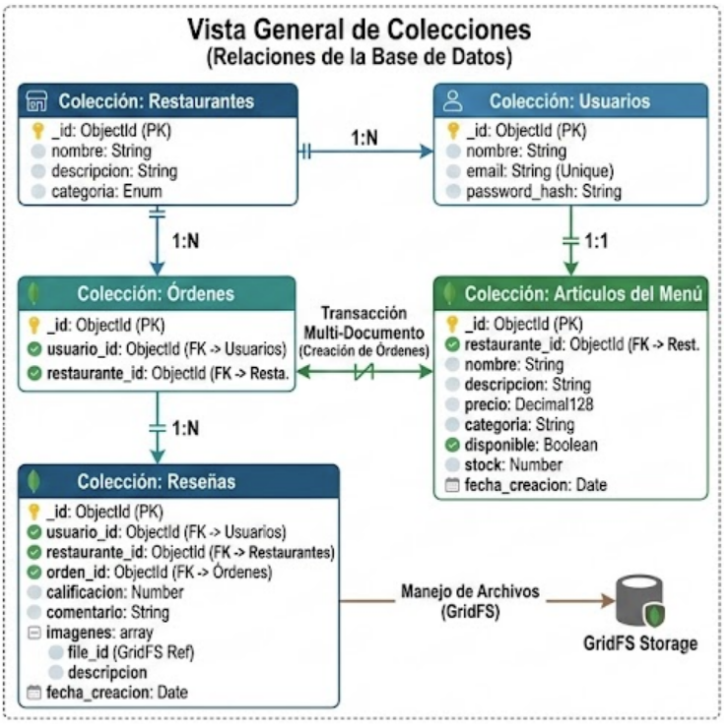

Ilustración 1. Vista General de Colecciones

a. Colección: Restaurantes

En la plataforma, esto representa a cada restaurante registrado.

```
{
 _id: ObjectId,
 nombre: String,
 descripcion: String,
 categoria: String,
 direccion: {
   type: "Point",
```

```
    coordinates: [Number, Number]  // [longitud, latitud]
  },
  telefono: String,
  email_contacto: String,
  rating_promedio: Number,
  total_resenas: Number,
  activo: Boolean,
  fecha_registro: Date
}
```

Ilustración 2. Documento Restaurante

Validaciones:

- nombre: requerido, string
- direccion: requerido, GeoJSON válido
- categoria: enum (Comida Rápida, Italiana, Asiática, etc.)
- rating_promedio: entre 0 y 5
- total_resenas: >= 0

Relaciones:

- 1 Restaurante a N Artículos del menú
- 1 Restaurante a N Órdenes
- 1 Restaurante a N Reseñas

Justificación:

- No se embeben los artículos del menú para evitar crecimiento excesivo
- Se almacena rating_promedio para evitar recalcular con agregaciones constantes

b. Colección: Usuarios

Representa a los clientes

```
{
  _id: ObjectId,
```

```
  nombre: String,
  email: String,
  password_hash: String,
  direccion_principal: String,
  telefono: String,
  rol: Enum,
  fecha_registro: Date,
  activo: Boolean
}
```

Ilustración 3. Documento Usuarios

Validaciones:

- email: requerido, único
- password_hash: requerido
- rol: enum (cliente, admin)

Relaciones:

- 1 Usuario a N Órdenes
- 1 Usuario a N Reseñas

Justificación:

- No se embeben órdenes por crecimiento ilimitado
- Modelo desacoplado facilita sharding futuro

c. Colección: Artículos del Menú

Se enfoca en un artículo perteneciente a un restaurante

```
{
  _id: ObjectId,
  restaurante_id: ObjectId,
  nombre: String,
  descripcion: String,
```

```
  precio: Number,
  categoria: String,
  disponible: Boolean,
  stock: Number,
  fecha_creacion: Date
}
```

Ilustración 4. Documento Artículo del Menú

Validaciones:

- restaurante_id: requerido
- precio: > 0
- stock: >= 0

Relaciones:

- N Artículos a 1 Restaurante
- 1 Artículo a N Órdenes

Justificación:

- Un restaurante puede tener cientos de artículos
- Se actualizan con frecuencia
- Evita documentos grandes y problemas de escritura

d. Colección: Órdenes

Documento central del sistema

```
{
  _id: ObjectId,
  usuario_id: ObjectId,
  restaurante_id: ObjectId,
  items: [
    {
      menu_item_id: ObjectId,
      nombre: String,
      cantidad: Number,
```

```
    precio_unitario: Number,
    subtotal: Number
  }
 ],
 estado: String,
 total: Number,
 metodo_pago: String,
 fecha_creacion: Date,
 fecha_actualizacion: Date
}
```

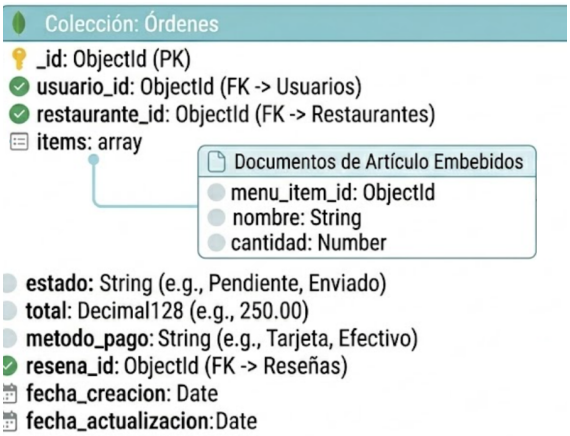

Ilustración 5. Documento Órdenes

Validaciones:

- items: array no vacío
- estado: enum (pendiente, preparando, enviado, entregado, cancelado)
- total: > 0

Relaciones:

- N Órdenes a 1 Usuario
- N Órdenes a 1 Restaurante
- 1 Orden a 1 Reseña

Justificación:

- items embebido porque pertenece exclusivamente a la orden.
- Se guarda nombre y precio_unitario para mantener consistencia histórica
- Evita lookups en consultas de historial

e. Colección: Reseñas

Guarda las evaluaciones de órdenes

```
{
 _id: ObjectId,
 usuario_id: ObjectId,
 restaurante_id: ObjectId,
 orden_id: ObjectId,
 calificacion: Number,
```

```
  comentario: String,
  imagenes: [
    {
      file_id: ObjectId,
      descripcion: String
    }
  ],
  fecha_creacion: Date
}
```

Ilustración 6. Documento Reseñas

Validaciones:

- calificacion: entre 1 y 5
- orden_id: requerido
- comentario: máximo 1000 caracteres

Relaciones:

- N Reseñas a 1 Usuario
- N Reseñas a 1 Restaurante
- 1 Reseña a 1 Orden

Justificación:

- Referencia a orden garantiza integridad lógica
- imagenes es multikey (soporta múltiples imágenes)
- Archivos reales almacenados en GridFS

4. Justificación de embedding vs referencing

a. Criterios de decisión

El diseño del modelo se basó en:

- Frecuencia de acceso conjunto de los datos
- Cardinalidad de la relación
- Crecimiento esperado del documento

- Necesidad de consistencia transaccional
- Impacto en rendimiento de lectura

b. Uso de Documentos Embebidos

Caso de ítems dentro de Órdenes

```
items: [
  {
    menu_item_id,
    nombre,
    cantidad,
    precio_unitario,
    subtotal
  }
]
```

Justificación: Los items, como documentos embebidos, se modelan. Esto dentro de orden porque tienen no tienen sentido fuera de ella. Cada vez que se consulta una orden se necesitan también sus productos. Adicionalmente, el embedding optimiza la lectura y evita operaciones "$lookup". Esto mejora el rendimiento en consultas frecuentes. Además, se almacenan campos como "nombre" y "precio_unitario" para preservar la consistencia histórica ante cambios de menú.

c. Uso de referencias

Tabla 2. Casos de referencias

Caso 1 Artículos del menú a Restaurante	**Caso 2** Órdenes a Usuario y Restaurante	**Caso 3** Reseñas a Usuario, Restaurante y Orden
Un restaurante puede tener cientos o miles de artículos	1 Usuario a N Órdenes; 1 Restaurante a N Órdenes	
Cardinalidad, actualización de precios y disponibilidad, escalabilidad	Las órdenes crecen de forma ilimitada, permite shard key basada en restaurante_id o fecha.	Reseñas independientes,agregaciones (promedios, rankings), reseña asociada a orden válida)

d. Manejo de Archivos (GridFS)

Las imágenes asociadas a reseñas no se almacenan directamente en el documento. Se utiliza: Referencia a file_id y archivos almacenados en GridFS. Esto porque las imágenes pueden exceder límites de tamaño, GridFS permite almacenamiento eficiente y distribuido y siempre se mantiene el documento liviano.

5. Funcionamiento general

El sistema será una API que gestione la persistencia, consulta y análisis de datos relacionados con restaurantes, pedidos y reseñas.

- Flujo de creación de pedido
    1. El usuario autenticado selecciona un restaurante.
    2. Consulta los artículos disponibles del menú.
    3. Selecciona uno o varios artículos indicando cantidades.
    4. El sistema:
        - Valida disponibilidad y stock.
        - Calcula subtotales por artículo.
        - Calcula el total del pedido.

    5. Se crea un documento en la colección Órdenes con:
        - Referencias a usuario y restaurante.
        - Arreglo embebido de items.
        - Estado inicial: pendiente.

    6. Se actualiza el stock de cada artículo del menú correspondiente.
    7. El pedido queda disponible para consulta por el restaurante y el usuario.

- Flujo de creación de reseña
    1. El usuario solo puede reseñar órdenes con estado entregado.
    2. El sistema valida que:
        - La orden exista.
        - Pertenezca al usuario.

    3. Se crea un documento en la colección Reseñas con:
        - Referencia a usuario, restaurante y orden.
        - Calificación y comentario.
        - Referencias a archivos almacenados en GridFS (si aplica).

    4. Se actualizan los campos:
        - rating_promedio
        - total_resenas
        - en la colección Restaurantes.

- Flujo de consultas analíticas

  El sistema permite:

    - Consultar restaurantes cercanos mediante índice geoespacial.

- Obtener restaurantes mejor calificados mediante aggregation.
- Analizar platillos más vendidos a partir del arreglo items en Órdenes.

6. Transacción Multi-Documento

Para garantizar consistencia en la creación de pedidos, se implementará una transacción multi-documento durante el proceso de compra.

- Caso: Creación de una orden

  La transacción incluye:
  - Inserción del documento en la colección Órdenes.
  - Actualización del campo stock en cada documento correspondiente en Artículos del Menú.

- Lógica de consistencia

- Si todos los artículos tienen stock suficiente:
  - Se inserta la orden.
  - Se descuentan las cantidades del stock.
  - Se confirma la transacción.
- Si algún artículo no tiene stock suficiente:
  - Se aborta la transacción.
  - No se inserta la orden.
  - No se modifica ningún stock.

7. Índices definidos y validación con explain()

Con base en los flujos principales (búsqueda por cercanía, consulta de menú por restaurante, historial de órdenes por usuario/restaurante y creación/consulta de reseñas), se proponen índices para cubrir los patrones de lectura más frecuentes y evitar colecciones escaneadas completas.

Colección Restaurantes:
- **Índice geoespacial** para consultas por proximidad (cercanos a un punto).
  - db.Restaurantes.createIndex({ direccion: "2dsphere" })
- **Índice para ranking/ordenamiento por rating** (cuando se muestre “mejor calificados” sin necesidad de aggregation).
  - db.Restaurantes.createIndex({ rating_promedio: -1, total_resenas: -1 })

Colección: **Usuarios**
- **Índice único** para login y evitar duplicados.
  - db.Usuarios.createIndex({ email: 1 }, { unique: true })

Colección: **ArticulosMenu** (Artículos del Menú)
- **Índice compuesto** para listar el menú de un restaurante y filtrar disponibilidad/stock.
  db.ArticulosMenu.createIndex({ restaurante_id: 1, stock: -1 })

- Opcional: si se filtra por “disponible” (flag boolean) en vez de stock, se puede ajustar a { restaurante_id: 1, disponible: 1 }.

Colección: **Ordenes**

- **Historial de órdenes por usuario con ordenamiento por fecha.**
  - db.Ordenes.createIndex({ usuario_id: 1, fecha_creacion: -1 })
- **Órdenes por restaurante (útil para analítica y panel del restaurante).**
  - db.Ordenes.createIndex({ restaurante_id: 1, fecha_creacion: -1 })
- **Filtro por estado dentro de un restaurante (pendiente/preparando/etc.).**
  - db.Ordenes.createIndex({ restaurante_id: 1, estado: 1, fecha_creacion: -1 })

Colección: Reseñas

- Reseñas por restaurante (listar y calcular métricas rápidamente).

```
db.Resenas.createIndex({ restaurante_id: 1, fecha_creacion: -1 })
```

- Unicidad lógica (una reseña por orden).

```
db.Resenas.createIndex({ orden_id: 1 }, { unique: true })
```

Validación con explain

La validación se realiza con explain("executionStats") buscando que el plan ganador utilice IXSCAN (y no COLLSCAN) y que totalDocsExamined sea cercano a nReturned.

Ejemplos de validación:

- Restaurantes cercanos (usa 2dsphere):

```
db.Restaurantes.find({
  direccion: {
    $near: {
      $geometry: { type: "Point", coordinates: [-90.5, 14.6] },
      $maxDistance: 3000
    }
  }
}).explain("executionStats")
```

- Menú por restaurante filtrando stock:

```
db.ArticulosMenu.find({
  restaurante_id: ObjectId("..."),
  stock: { $gt: 0 }
}).explain("executionStats")
```

- Historial de órdenes por usuario (ordenado por fecha):

```
db.Ordenes.find({ usuario_id: ObjectId("...") })
  .sort({ fecha_creacion: -1 })
  .explain("executionStats")
```

- Reseñas por restaurante (ordenadas por fecha):

```
db.Resenas.find({ restaurante_id: ObjectId("...") })
  .sort({ fecha_creacion: -1 })
  .explain("executionStats")
```

8. Aggregation Pipelines

Las consultas analíticas se resuelven con pipelines sobre Órdenes y Reseñas, principalmente para ranking y para "top vendidos", aprovechando que items está embebido en el documento de Órdenes.

a. Restaurantes mejor calificados (a partir de Reseñas): Objetivo: obtener el top de restaurantes por calificación promedio y cantidad de reseñas, evitando depender únicamente de campos pre-calculados.

```
db.Resenas.aggregate([
  { $group: {
      _id: "$restaurante_id",
      rating_promedio: { $avg: "$calificacion" },
      total_resenas: { $sum: 1 }
  }},
  { $sort: { rating_promedio: -1, total_resenas: -1 } },
  { $limit: 10 },
  { $lookup: {
      from: "Restaurantes",
      localField: "_id",
      foreignField: "_id",
      as: "restaurante"
  }},
  { $unwind: "$restaurante" },
  { $project: {
      _id: 0,
      restaurante_id: "$_id",
      nombre: "$restaurante.nombre",
      categoria: "$restaurante.categoria",
      rating_promedio: { $round: ["$rating_promedio", 2] },
      total_resenas: 1
  }}
])
```

b. Platillos más vendidos (a partir del arreglo items en Órdenes):

i. Objetivo: ranking de productos por cantidad vendida y monto generado, explotando items embebido.

```
db.Ordenes.aggregate([
  { $match: { estado: "entregado" } },
  { $unwind: "$items" },
  { $group: {
      _id: "$items.menu_item_id",
      nombre: { $first: "$items.nombre" },
      unidades_vendidas: { $sum: "$items.cantidad" },
      ingreso_total: { $sum: "$items.subtotal" }
  }},
  { $sort: { unidades_vendidas: -1, ingreso_total: -1 } },
  { $limit: 10 },
  { $project: {
      _id: 0,
      menu_item_id: "$_id",
      nombre: 1,
      unidades_vendidas: 1,
      ingreso_total: { $round: ["$ingreso_total", 2] }
  }}
])
```

c. Ventas por restaurante (monto total y número de órdenes):métrica simple para analítica del lado del restaurante.

```
db.Ordenes.aggregate([
  { $match: { estado: "entregado" } },
  { $group: {
      _id: "$restaurante_id",
      total_ordenes: { $sum: 1 },
      ventas_totales: { $sum: "$total" }
  }},
  { $sort: { ventas_totales: -1 } },
  { $limit: 10 },
  { $lookup: {
      from: "Restaurantes",
      localField: "_id",
      foreignField: "_id",
      as: "restaurante"
  }},
  { $unwind: "$restaurante" },
  { $project: {
      _id: 0,
      restaurante_id: "$_id",
      nombre: "$restaurante.nombre",
      total_ordenes: 1,
      ventas_totales: { $round: ["$ventas_totales", 2] }
  }}
])
```

9. Shard Key y estrategia de escalabilidad

Shard Key propuesta: restaurante_id

- Justificación
    - Las órdenes crecen proporcionalmente al número de restaurantes.
    - Distribuye datos entre shards según restaurante.
    - Permite consultas eficientes por restaurante (historial, ventas, analítica).
    - Reduce hotspots comparado con una shard key monotónica como fecha.

  Alternativamente, podría utilizarse usuario_id si el crecimiento se concentra en grandes volúmenes por cliente.

  La colección Órdenes es la candidata natural a sharding debido a:
    - Alto volumen de escrituras.
    - Alto volumen de consultas analíticas.
    - Crecimiento continuo.